

WHEN THE SPACE IS CONTINUOUS AND QUERIES ARE SCARCE

Bayesian Optimization

Model the whole space from a handful of points —
then ask, each time, the one question that teaches you
the most.

LECTURERS

Thomas Langerak · Mark Colley

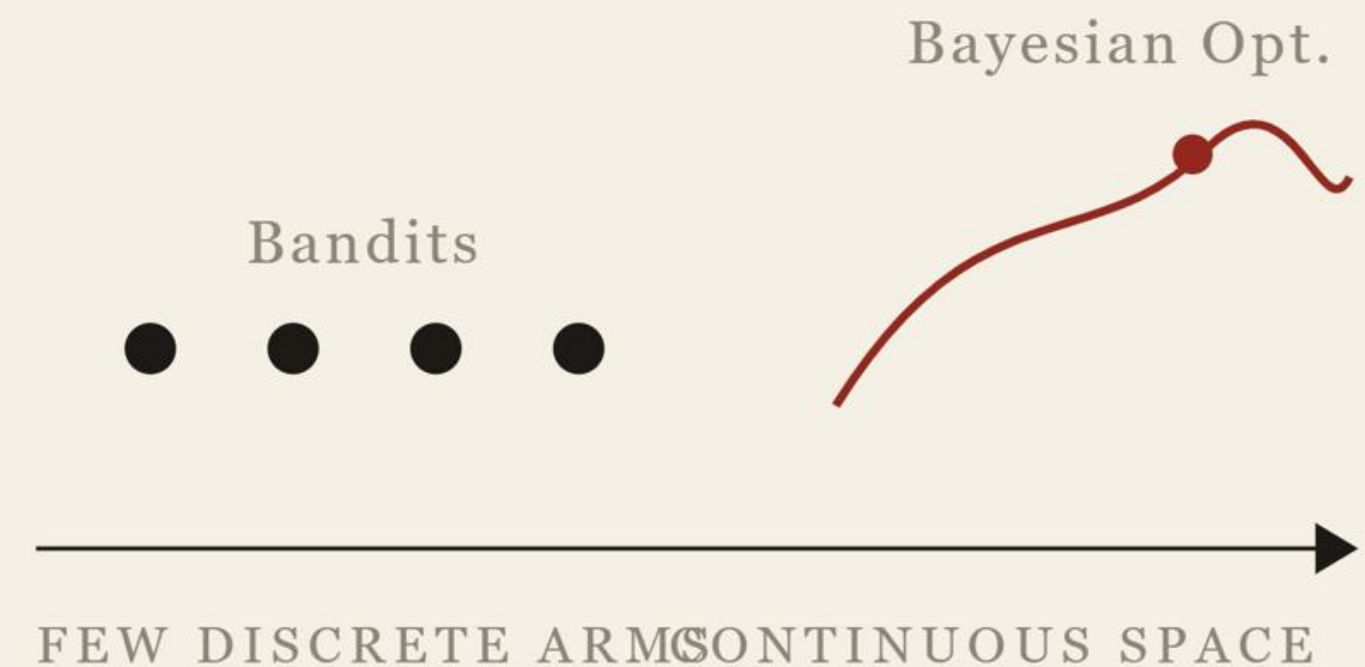
$$x_{t+1} = \arg \max_x \alpha(x)$$

From a fixed menu to an infinite continuum.

The Foundation lecture framed everything as *maximize a hidden utility with few, noisy queries*. Today we keep the loop but let the design itself become a **vector of continuous knobs** — so there is no list to enumerate.

RECALL

Propose $\rightarrow$ query $\rightarrow$ observe $\rightarrow$ update. BO is one concrete way to do the *propose* and *update* steps when $\mathcal{X}$ is continuous.



Optimize an expensive, black-box function in very few evaluations.

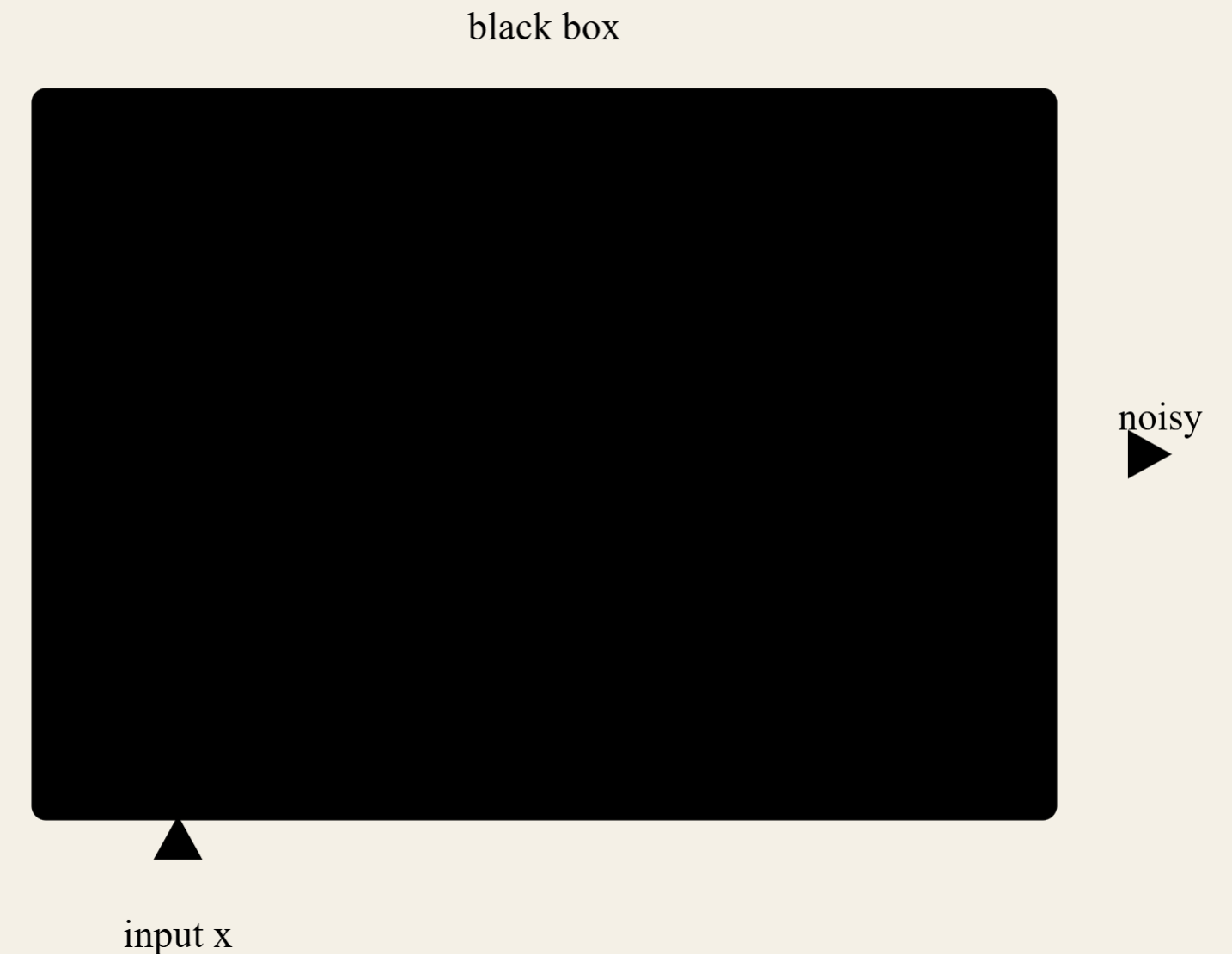
No gradients, no formula — each evaluation costs a real user query. $\mathcal{X}$ is continuous and infinite, so we can't try every option. We need to *generalize* from the points we've seen.

∞

candidate designs

~ 20

queries we can afford



The usual tools all assume queries are cheap. Ours aren't.

GRID SEARCH

Curse of dimensionality

A grid of 10 per axis is 10^d points — 100 000 in five dimensions. Hopeless on a budget of twenty.

RANDOM SEARCH

Ignores the past

Never learns. Each draw is blind to every answer before it — it wastes the very information we paid for.

GRADIENT DESCENT

Needs a gradient

Black box: no derivative. Finite differences cost extra queries per step, and noise swamps the estimate.

THE BET

Spend a little computation to *model* the function so that every expensive query is chosen, not stumbled into. BO trades cheap CPU cycles for scarce human answers.

Tuning a notification's haptic buzz to feel "right."

Two continuous knobs: *intensity* and *duration*. There is no formula for "pleasant" — only this user, rating each buzz they feel. Every rating costs their attention, so we get maybe fifteen.

$\mathbf{x}$ $[0,1] \times [0,1]$ — the two knobs $u(\mathbf{x})$ how pleasant, in this person's head

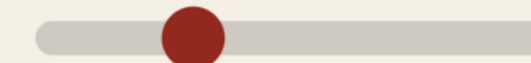
y a noisy 1–7 rating

We'll return to this buzz on every slide — watch the belief sharpen as ratings arrive.

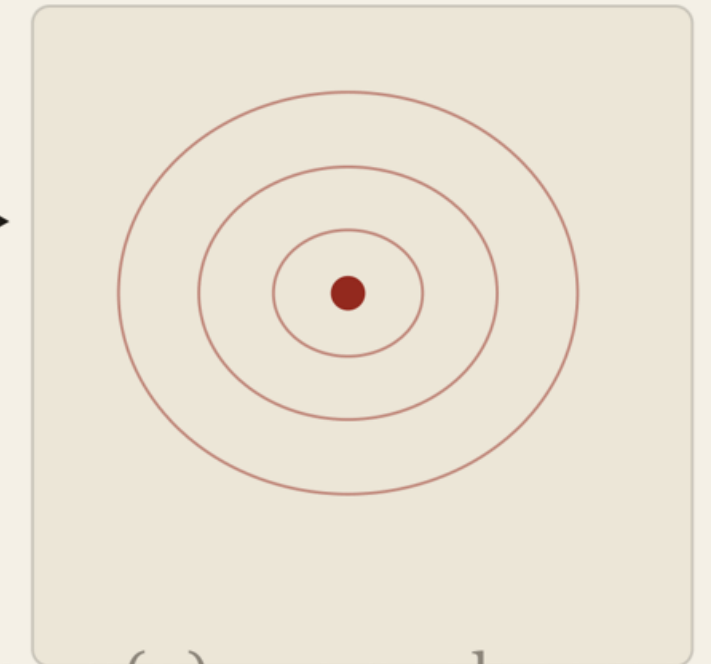
DESIGN X



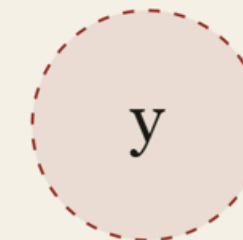
intensity



duration



USER hidden $u(\mathbf{x})$ — a peak somewhere



THE WHOLE METHOD IN TWO PARTS

A model of the space, and a rule for where to ask.

1 · SURROGATE

A Gaussian process:
prediction + uncertainty
everywhere

2 · ACQUISITION

A function that scores how
useful each next query is

A cheap, trainable stand-in for the function we can't afford to probe.

Surrogate model (a.k.a. response surface, emulator) — a statistical model $\hat{g}(x)$ fit to the answers seen so far that *approximates the true $f(x)$* everywhere, and is cheap enough to interrogate as often as we like.

Each real query costs a human answer; the surrogate costs nothing. So we shift all the searching onto the model and spend real queries only where the model says it's worth it.

THE KEY A plain regressor gives only point 2 — a prediction. A surrogate for BO *must* also give calibrated uncertainty, because the acquisition step decides where to look using *both* μ and σ . Our choice of surrogate, next, is the **Gaussian process**.

THREE THINGS A SURROGATE MUST DO

1 · PREDICT

Interpolate the unseen

Give a best guess $\mu(x)$ at every x , generalizing from the handful of observed points.

2 · QUANTIFY DOUBT

Report its own uncertainty

Return $\sigma(x)$ — confident near data, unsure far from it. This is what makes *exploration* possible.

3 · UPDATE CHEAPLY

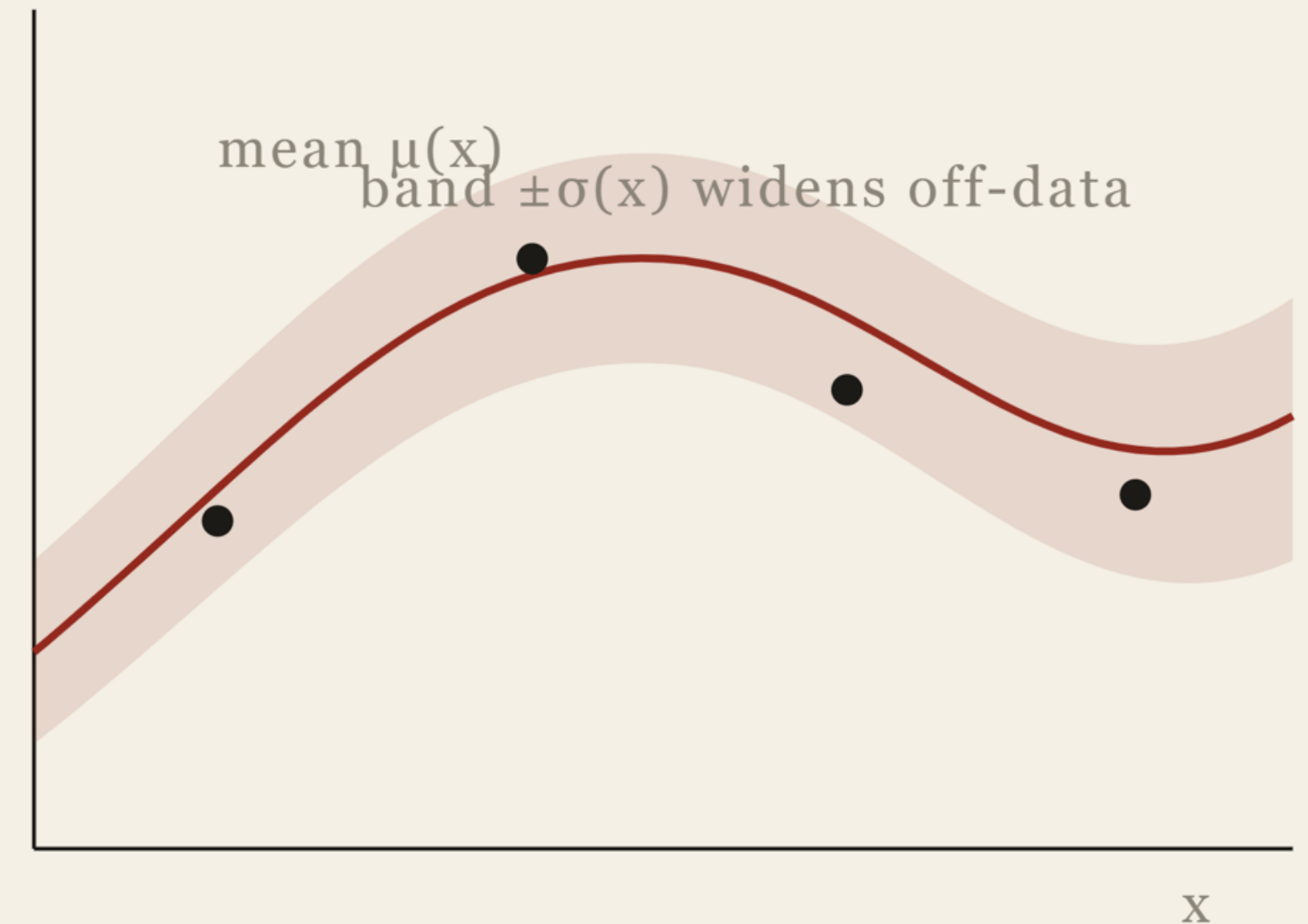
Refit as answers arrive

Fold each new (x,y) back in without re-touching the expensive function.

A distribution over functions, not over numbers.

A **GP** gives, at every input x , a predicted mean $\mu(x)$ and a variance $\sigma^2(x)$. Near data it is confident; far from data it is unsure. The *kernel* encodes our belief about smoothness — how far one answer informs its neighbours.

$$f(x) \sim \mathcal{GP}(\mu(x), k(x, x'))$$



A GP says any finite set of points is *jointly Gaussian*. Stack the observed values f and a test point f_* into one Gaussian, then **condition** on what we saw. That single step of Gaussian algebra is the entire surrogate.

$$\begin{bmatrix} \mathbf{f} \\ f_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} K + \sigma_n^2 I & \mathbf{k}_* \\ \mathbf{k}_*^\top & k_{**} \end{bmatrix}\right)$$

condition on the observed $\mathbf{y}$ ↓

$$\mu(x_*) = \mathbf{k}_*^\top (K + \sigma_n^2 I)^{-1} \mathbf{y}$$

$$\sigma^2(x_*) = k_{**} - \mathbf{k}_*^\top (K + \sigma_n^2 I)^{-1} \mathbf{k}_*$$

$\mathbf{K}$ kernel matrix between observed points, $K_{ij}=k(x_i, x_j)$

$\mathbf{k}_*$ kernel vector from the test point to each observation

k_{**} prior variance at the test point, $k(x_*, x_*)$

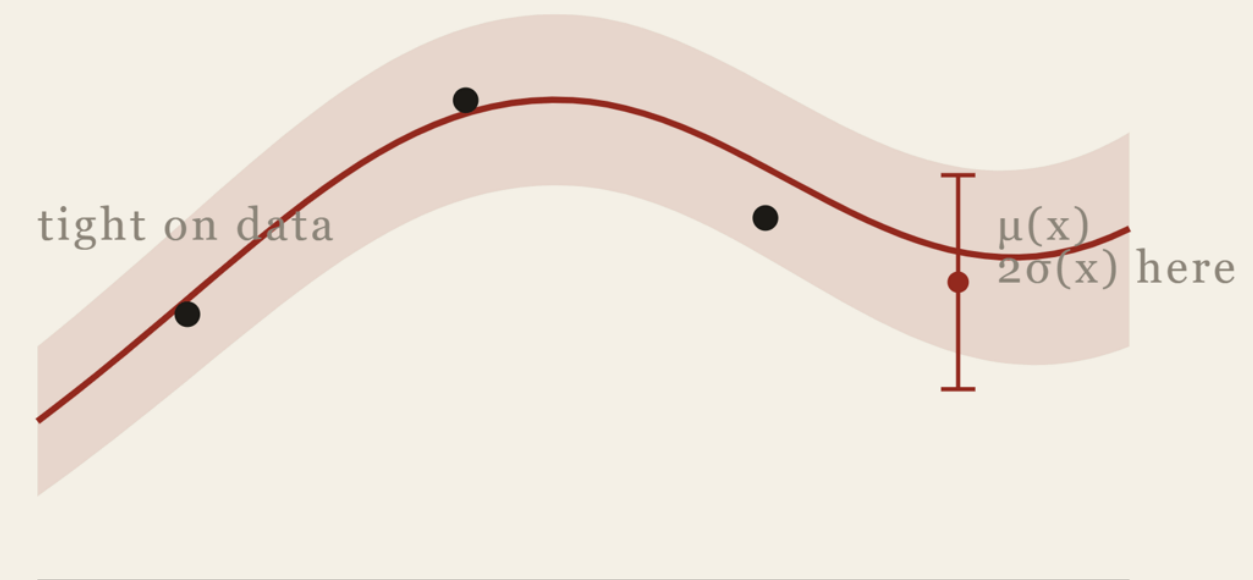
σ_n^2 observation-noise variance — softens the fit

READ IT

The mean is a *weighted average of the answers* ; the variance is the prior minus what the data explains. Far from data, $k_* \rightarrow 0$, so $\sigma^2 \rightarrow k_{**}$ — full uncertainty.

Prediction *and* calibrated doubt.

Most models give a point estimate. The GP hands back a full Gaussian at every x : a mean to exploit and a standard deviation to explore. The acquisition function in Part 2 lives entirely off these two numbers.

**COST**

Fitting the GP — cost grows as $O(n^3)$ in the number of observations.
That only bites at thousands of points
Human-in-the-loop budgets are tens of evaluations (fatigue, recruitment cost)
At $n \approx 30$ the refit takes microseconds — the human is the bottleneck, not the CPU

Choosing the kernel is choosing what "smooth" means.

SMOOTH

RBF / Gaussian

Infinitely differentiable — assumes the utility surface is very smooth. The common default.

FLEXIBLE

Matérn

A roughness dial. Matérn-5/2 is the workhorse for real, slightly bumpy functions.

LENGTHSCALE

How far news travels

Short → wiggly, each point informs little. Long → broad, one point speaks for a wide region.

Get the kernel wrong and the surrogate misleads every decision that follows — it is the single most consequential modelling choice.

09 THE TWO KERNELS YOU'LL ACTUALLY USE

RBF / squared-exponential

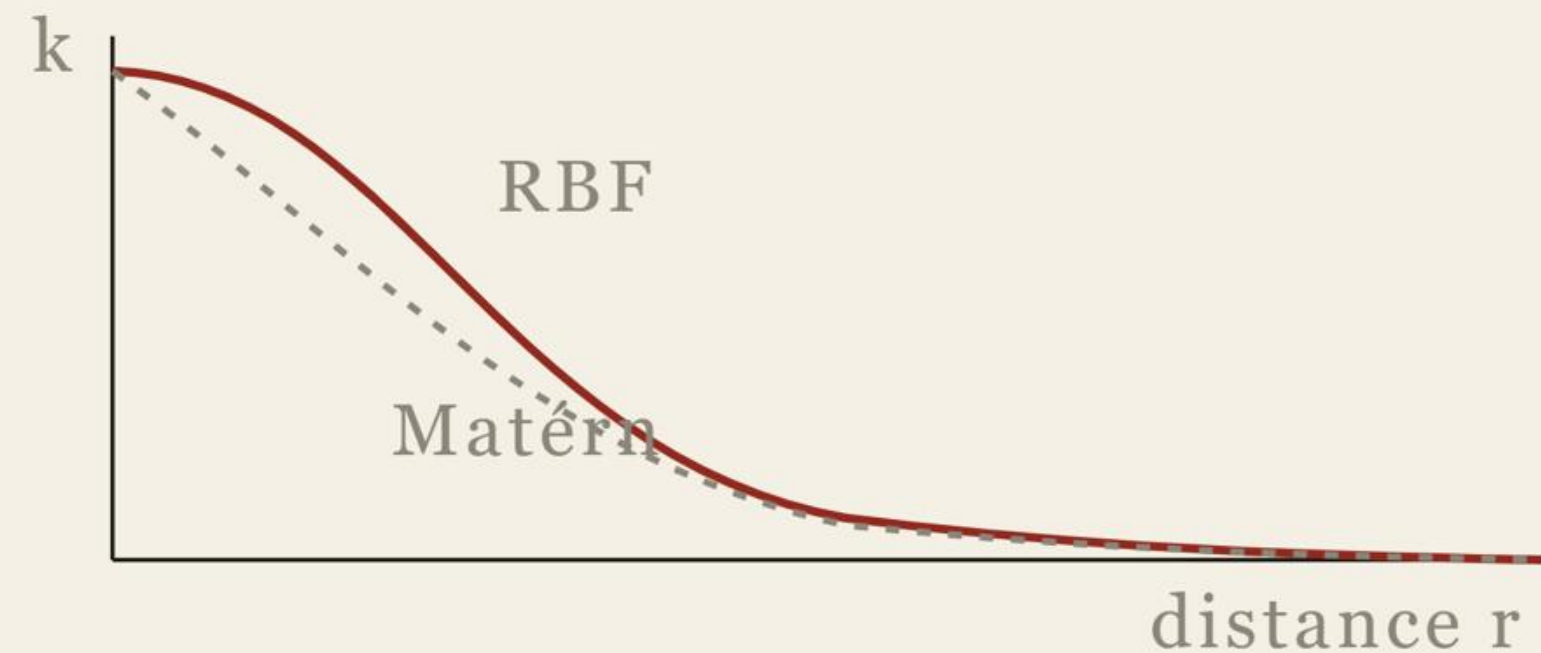
$$k(x, x') = \sigma_f^2 \exp\left(-\frac{\|x-x'\|^2}{2\ell^2}\right)$$

Matérn-5/2 ($r = \|x-x'\|$)

$$k(r) = \sigma_f^2 \left(1 + \frac{\sqrt{5}r}{\ell} + \frac{5r^2}{3\ell^2}\right) \exp\left(-\frac{\sqrt{5}r}{\ell}\right)$$

ℓ lengthscale — reach of correlation σ_f^2 signal variance — vertical scale

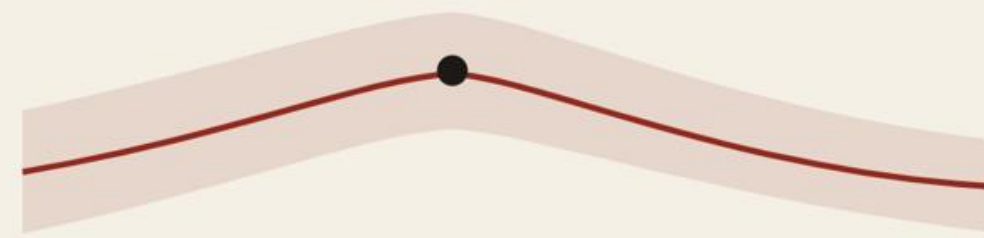
Both depend only on the *distance* between points. RBF (Radial Basis Function) makes draws infinitely smooth — often too smooth for real preferences. **Matérn-5/2** permits gentle kinks and is the field default e.g., in BoTorch.



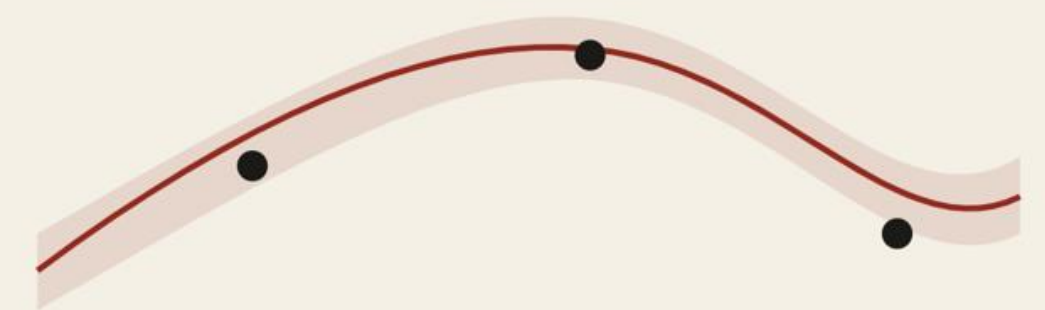
Start with a flat prior; each rating carves the band.



PRIOR — NO DATA, FLAT



1 RATING — PINCHED THERE



3 RATINGS — A PEAK FORMS

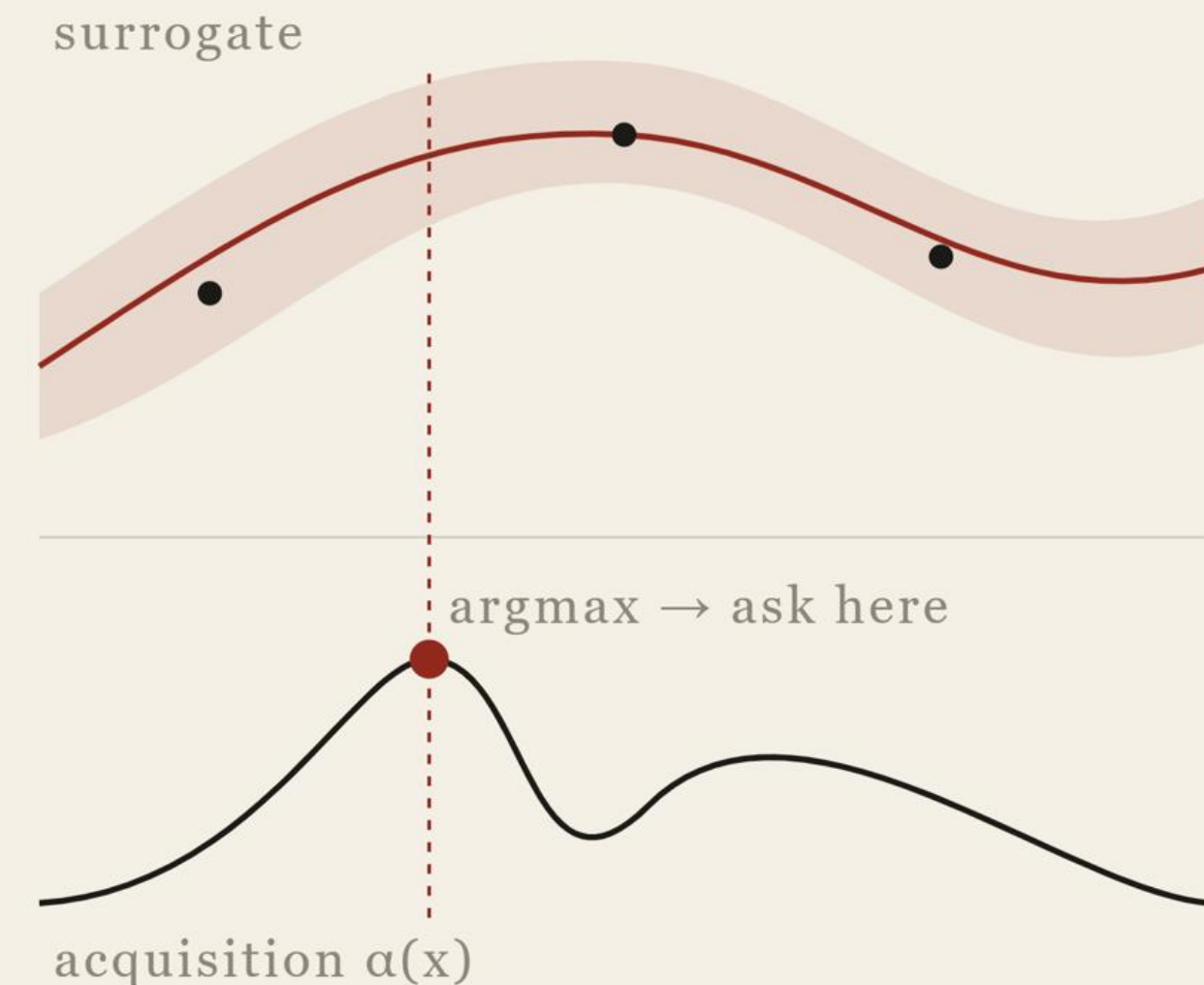
The prior is our smoothness assumption made visible; the posterior is that assumption bent to fit the data.

Turn the belief into a single decision: where to ask next.

The acquisition function $\alpha(x)$ scores every candidate by how useful a query there would be — high where the mean is high (*exploit*) and where uncertainty is high (*explore*). We query its maximum.

$$x_{t+1} = \arg \max_{x \in \mathcal{X}} \alpha(x; \mathcal{D}_t)$$

Note the trick: we replaced an expensive search over f with a cheap search over α .



Exploit the known peak, or explore the unknown valley?

Two candidate buzzes. One sits on the current best mean — safe, incremental. The other sits where the band is widest — risky, but it might hide a far better setting. Spend a query on the wrong one and it's gone for good.

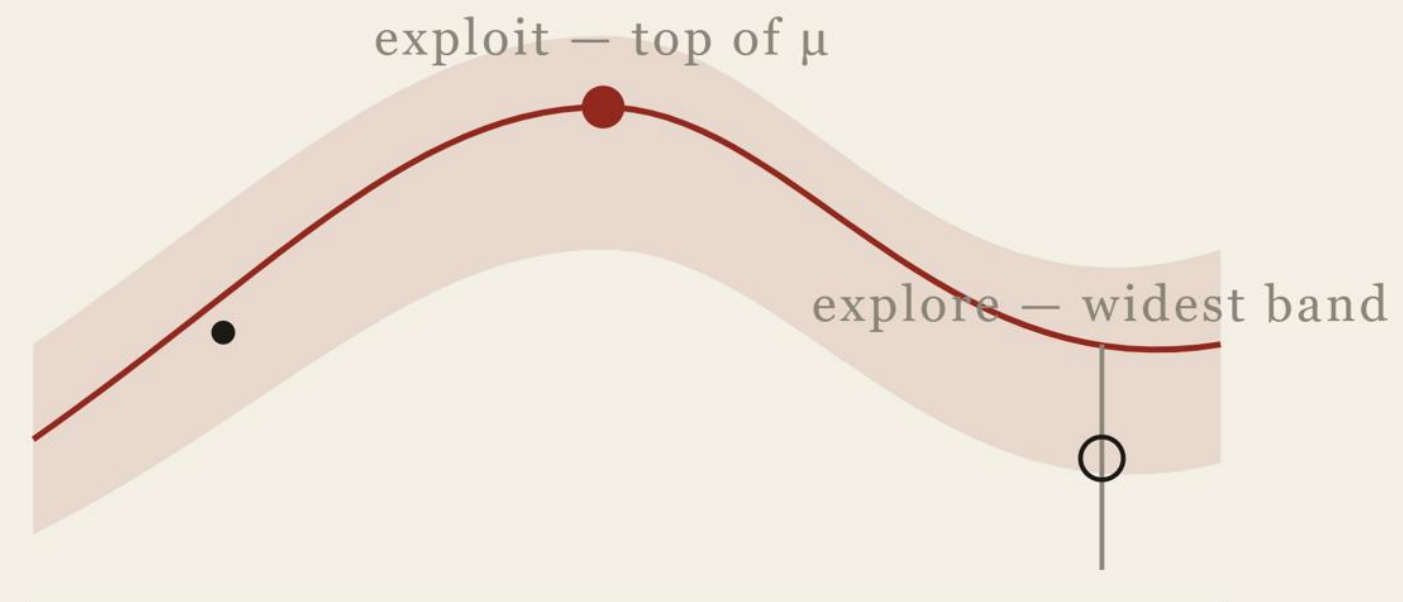
EXPLOIT

high $\mu(x)$ — refine what already works

EXPLORE

high $\sigma(x)$ — reduce what we don't know

An acquisition function is just a *recipe for combining μ and σ* into one score. The next four slides are four recipes.



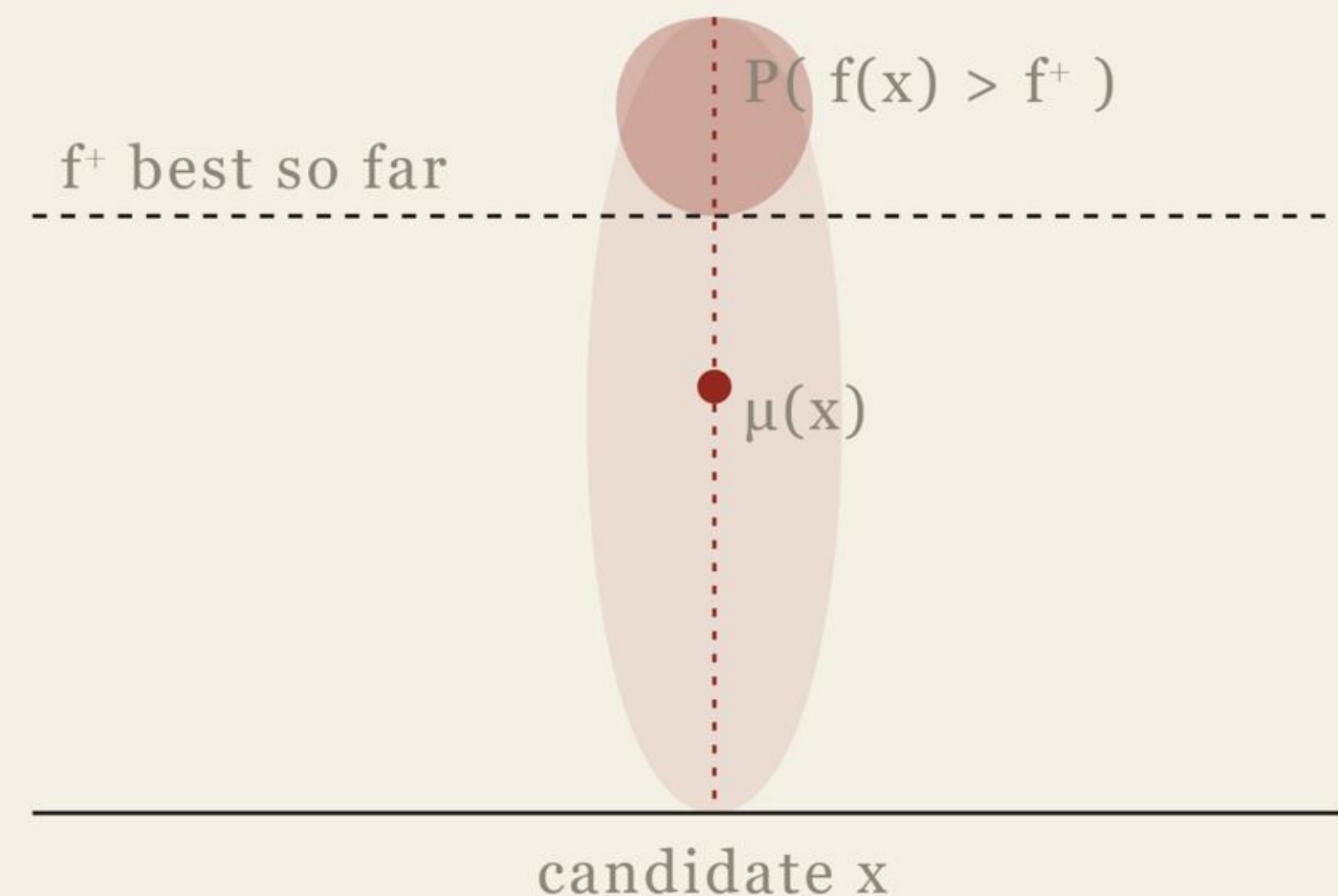
"How likely is this point to beat my best so far?"

Let f^+ be the best rating observed. At x the prediction is Gaussian, so the chance it exceeds f^+ is just the tail above that line — a closed form via the standard-normal CDF (cumulative distribution function) Φ .

$$\alpha_{\text{PI}}(x) = \Phi\left(\frac{\mu(x) - f^+ - \xi}{\sigma(x)}\right)$$

PITFALL

PI is greedy — it counts *any* improvement equally, so it clusters around the known peak. The margin ξ buys some exploration, but it's a knob you must tune.



PI asks *whether* we improve. EI asks *by how much* — the expected size of the improvement $I = \max(0, f(x) - f^+)$.

$$\alpha_{\text{EI}}(x) = \mathbb{E}[\max(0, f(x) - f^+)]$$

with $z = \frac{\mu(x) - f^+ - \xi}{\sigma(x)} \downarrow$

$$\alpha_{\text{EI}}(x) = \underbrace{(\mu - f^+ - \xi) \Phi(z)}_{\text{exploit}} + \underbrace{\sigma(x) \phi(z)}_{\text{explore}}$$

Φ standard-normal CDF — the improvement's probability

ϕ standard-normal PDF — rewards raw uncertainty

ξ exploration margin, often ≈ 0.01

WHY DEFAULT

The two terms *are* exploit + explore, fused automatically with no β to set. Closed-form, differentiable, and well-behaved — start here.

Certainty kills EI; the margin ξ tilts it toward exploring.

EDGE CASE

$$\sigma(x) \rightarrow 0 \Rightarrow EI \rightarrow 0$$

A point we're already sure about is never worth a query — EI won't re-ask what it knows.

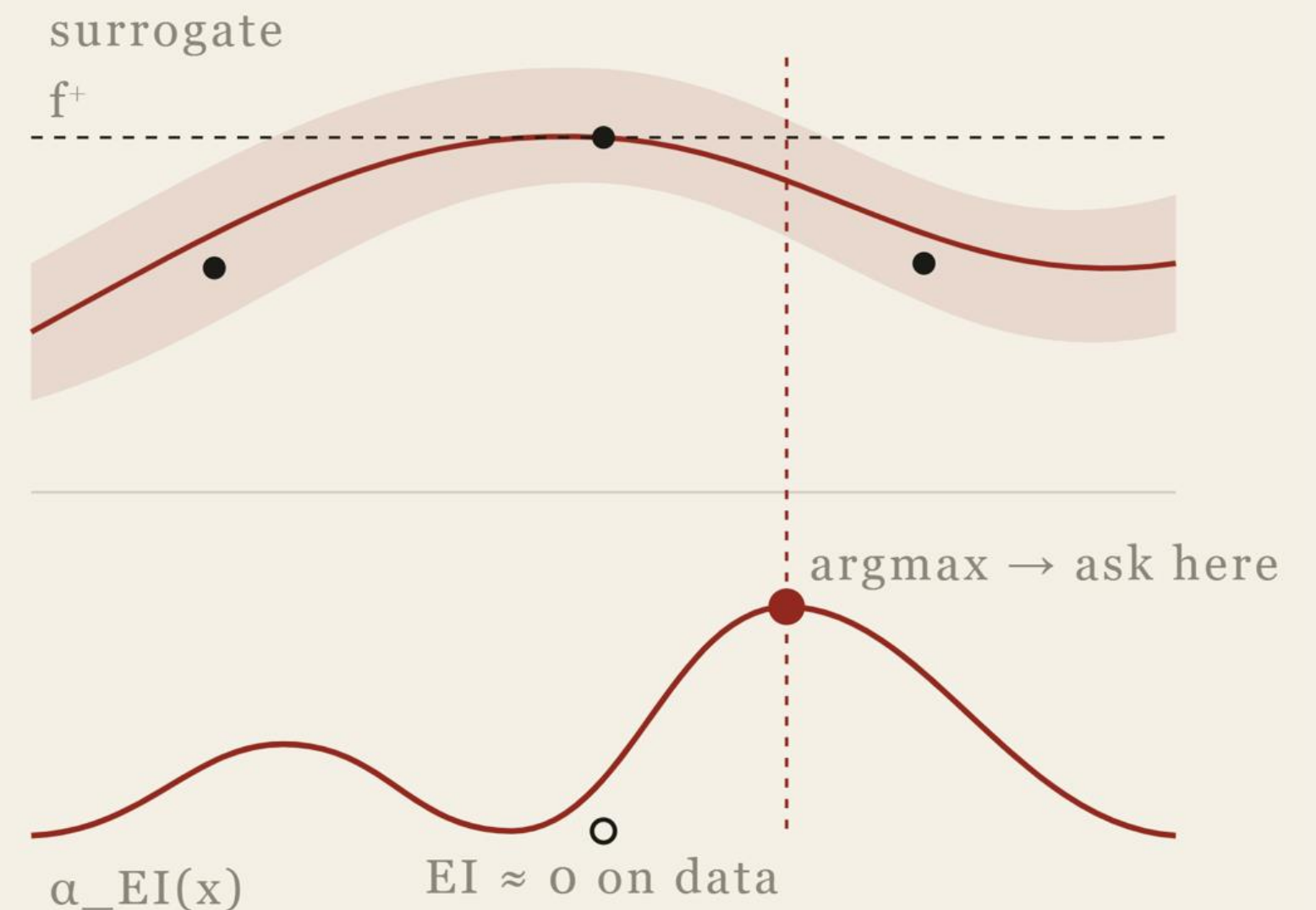
THE KNOB ξ

Raise it to demand a bigger win

ξ requires a candidate to clear f^+ by a margin before it counts — pushing into less-certain ground. $\xi \approx 0.01$ is the usual light touch.

CAVEAT

Standard EI assumes a noiseless f^+ ; with noisy ratings, use the incumbent's *posterior mean* (or "noisy EI").

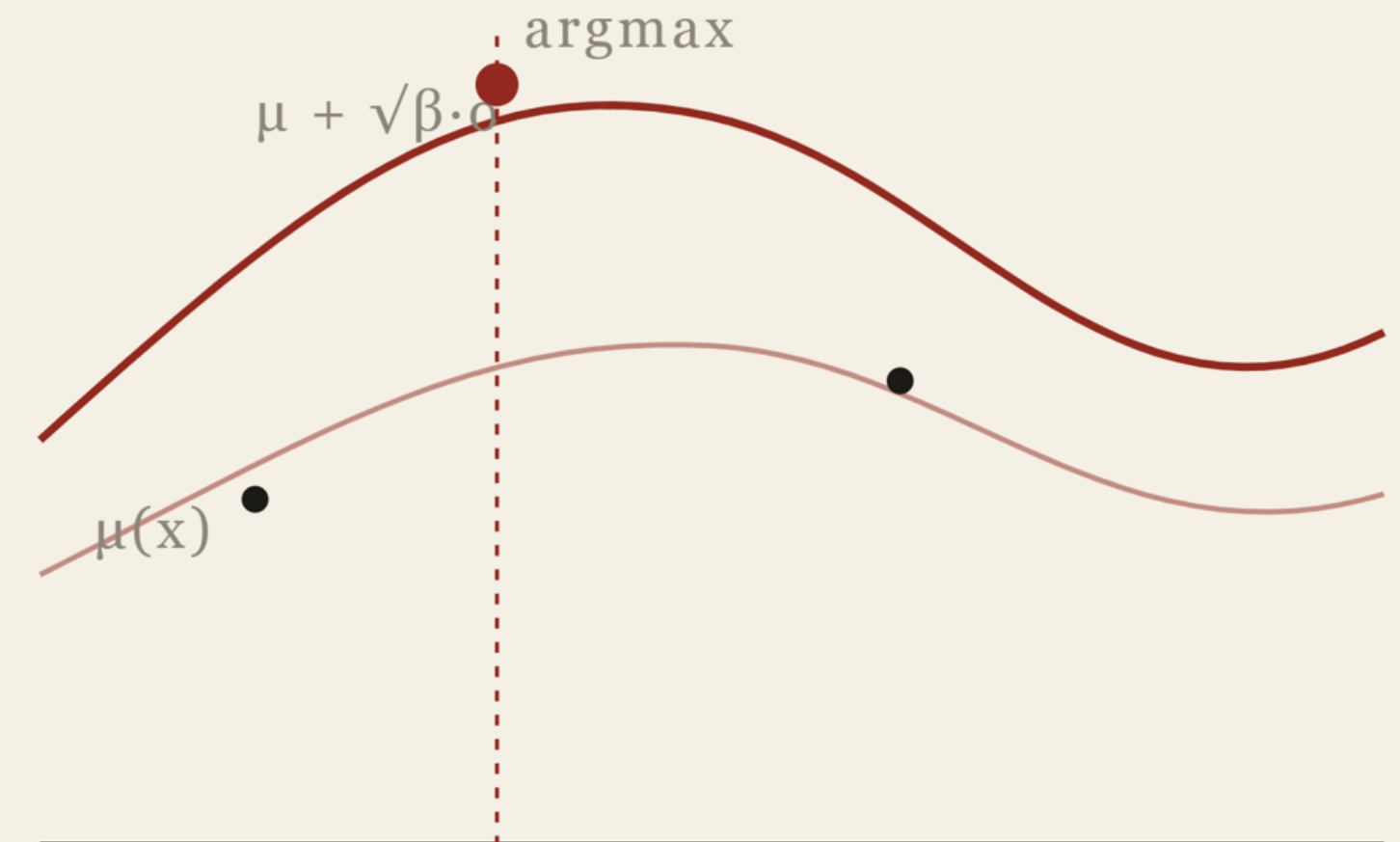


Optimism: judge each point by its best plausible value.

Add a multiple of the std-dev to the mean and maximize that. The dial β is the explore–exploit trade made explicit: small β exploits, large β explores.

$$\alpha_{\text{UCB}}(x) = \mu(x) + \sqrt{\beta_t} \sigma(x)$$

GUARANTEE With a schedule for β_t , GP-UCB is provably *no-regret* — average regret $\rightarrow 0$. The rare acquisition rule with a real bound.

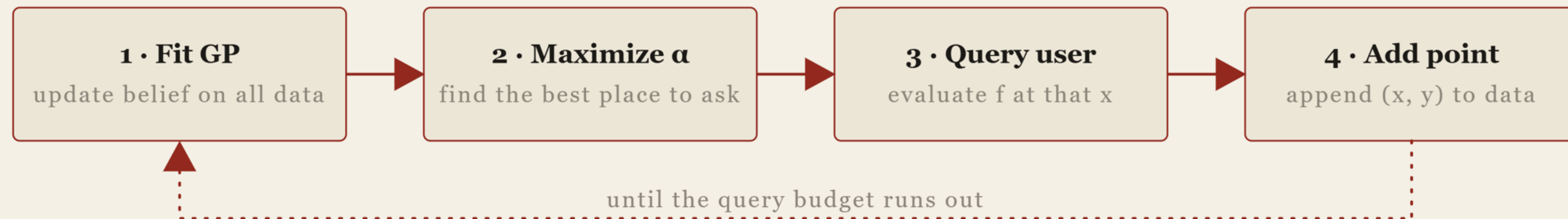


ACQUISITION	SCORE	CHARACTER	REACH FOR IT
Prob. of Improvement	$\Phi(z)$	Greedy, exploit-heavy	quick, low-noise refinement
Expected Improvement	$(\mu - f^*)\Phi(z) + \sigma\phi(z)$	Balanced, no tuning	the default first choice
Upper Conf. Bound	$\mu + \sqrt{\beta} \cdot \sigma$	Tunable, has regret bound	you want a guarantee / a dial
Thompson	argmax of a sample	Randomized, parallel	batch / many in parallel
Entropy · MES	info gain on x^*/f^*	Most efficient, costly	queries are extremely precious

USUALLY USED: qLogNoisyExpectedImprovement

Same surrogate, different question — each acquisition encodes a different notion of "useful."

Fit, optimize the acquisition, query, repeat.



The first few points aren't chosen — they're spread.

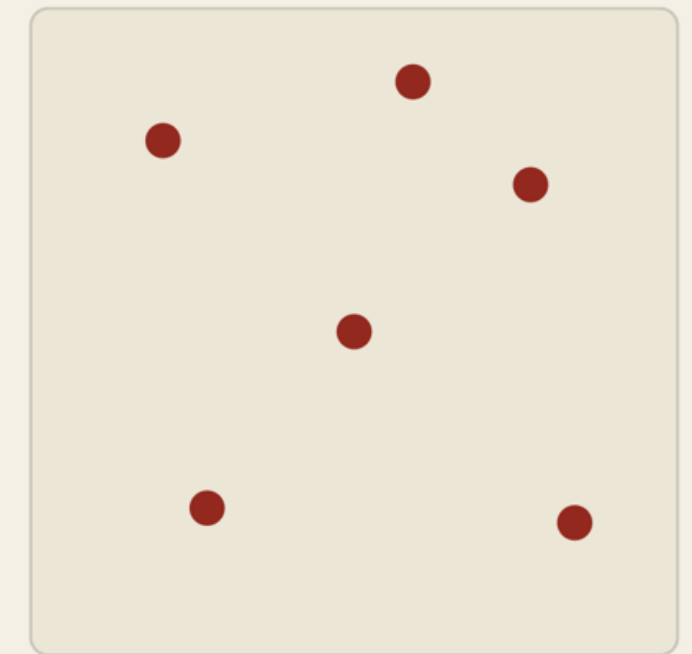
With no data, every acquisition rule is blind. So we start with a small *space-filling* design — Sobol or Latin-hypercube samples — to give the GP something to fit before it can reason. A common split: a handful to seed, the rest run by the loop.

why not random random clumps and leaves gaps; Sobol covers evenly

rule of thumb seed with a few points per dimension, then optimize

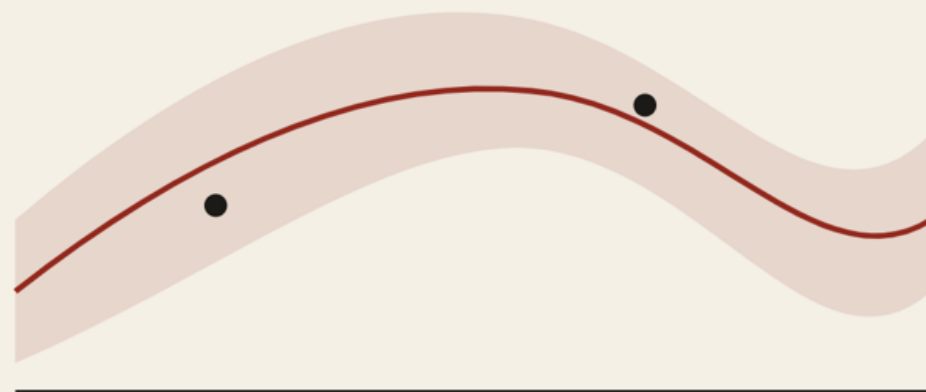


random — clumps & gaps

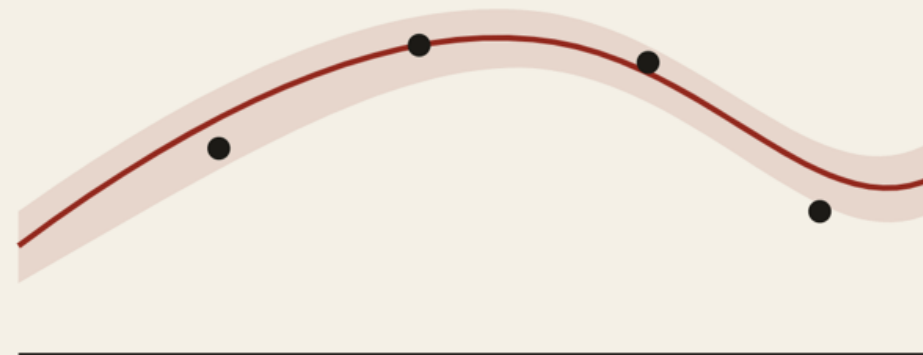


Sobol — even cover

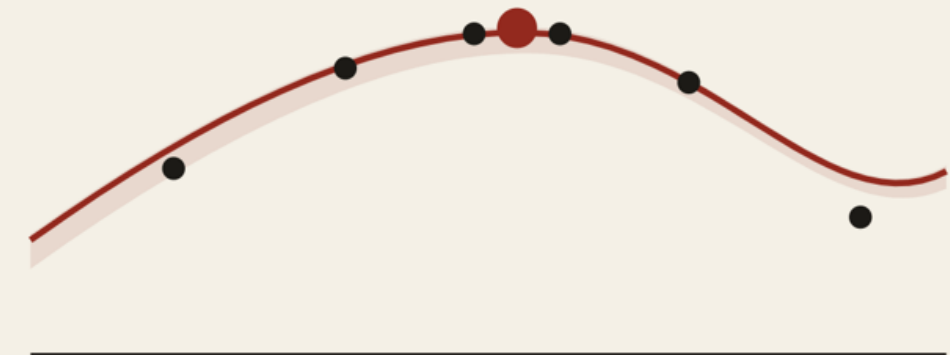
Each query tightens the belief near the peak.



AFTER 3 QUERIES — UNSURE



AFTER 6 QUERIES — NARROWING



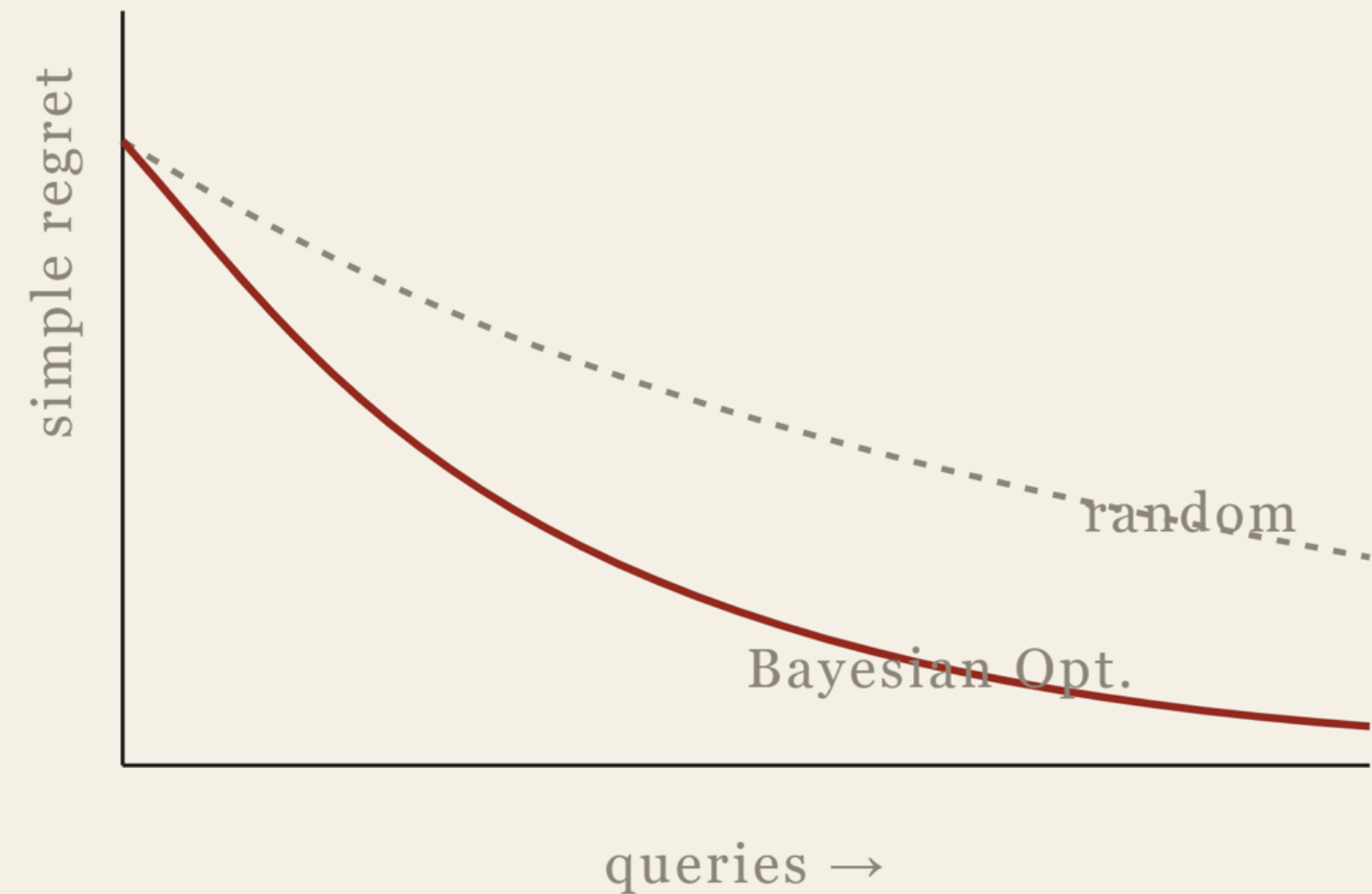
AFTER 10 — OPTIMUM LOCATED

Track simple regret — the gap to the true best.

After T queries, **simple regret** is how far the best design found still falls short of the unknown optimum. Plot it against query count and you get the curve that earns BO its keep.

$$r_T = f(x^*) - \max_{t \leq T} f(x_t)$$

In a real study you can't see $f(x^*)$ — so we benchmark on synthetic functions, or report best-rating-so-far against random search.





FROM THE WHITEBOARD TO THE LAB

BO in the wild

The loop is elegant on paper. Now: where has it actually optimized a real interface for a real person — and what bites you when you try?

Whenever a few continuous knobs meet an expensive human judgement.

Visual & sound design

Tune colour, contrast, timing, or audio parameters to taste in a handful of trials.

Haptics & hardware

Find vibration or actuation settings that feel right without exhaustive sweeps.

Interaction parameters

Gain, sensitivity, thresholds, animation curves fitted per user.

Hyperparameters

The classic: tuning expensive models or simulations where each run is costly.

CHI 2021 Kadner et al.

AdaptiFont

BO searches a generative font model's latent space to *maximize each reader's measured reading speed* — no rating asked. The behaviour is the query, and the font adapts to the person.

Signal: words-per-minute, implicit.

CHI 2016 Khajah, Roads, Lindsey, Liu, Mozer

Designing Engaging Games

BO tunes game-design parameters across players to *maximize total time played* — engagement as the objective. It rediscovered, and beat, expert-set defaults.

Signal: engagement, aggregated.

The "query" need not be a rating — any measurable response that tracks utility will do.

1 Normalize your inputs to the unit cube.

A single lengthscale can't serve "intensity 0–1" and "duration 0–500 ms" at once. Rescale every axis, or use a separate lengthscale per dimension (ARD).

2 Estimate the noise — don't assume it's zero.

Human answers are noisy; force $\sigma_n=0$ and the GP will contort through every point. Fit it, or set a sensible floor.

3 Default to Matérn-5/2 + EI, then revisit.

A safe, well-tested starting pair. Switch to UCB if you want an explicit explore dial, or Thompson if you need batches.

4 Watch for over-exploitation.

If queries pile onto one spot, the surrogate is over-confident — raise exploration (ξ or β), or re-fit hyperparameters less aggressively early on.

1 Dimensionality.

GPs degrade past roughly 10–20 dimensions — the space is too empty for a few points to cover.

2 People give comparisons, not numbers.

Absolute ratings are noisy and drift; users are far more reliable saying "A > B."

3 One query at a time is slow.

Often we want to test a whole batch in parallel within one user session.